

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO3: Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)

Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks:100

Annexure A

EXPERIMENTS

Code of Conduct:

*I **B. Santhosh Reddy (192411186)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Q. No	Understanding of Concepts (25)	Experimental Design (30)	Use of Tools (20)	Analysis of Results (15)	Documentation (10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 30	/ 20	/ 15	/ 10	/ 100

EXPERIMENT 1

Recognition and Counting of Uppercase Letters

Aim

To design and implement a program that recognizes uppercase letters from **A to Z** in a given input string and calculates the total number of uppercase letters.

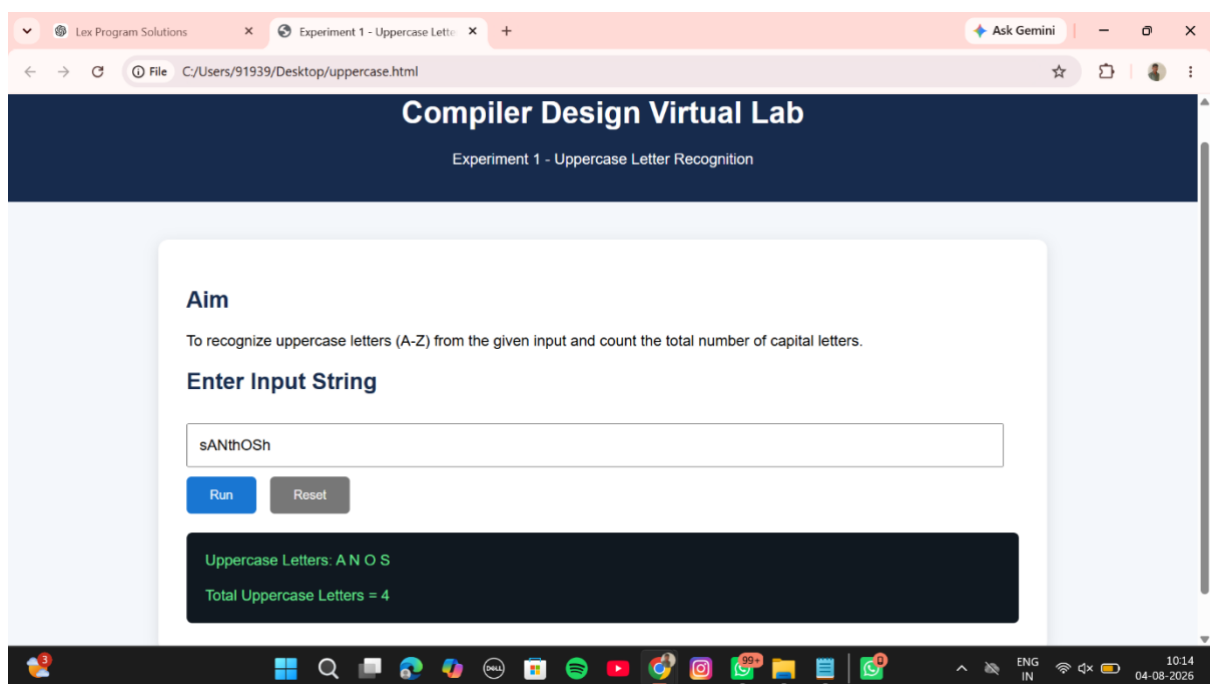
Problem Statement

The program accepts a string from the user and examines every character. If a character belongs to the range A-Z, it is recognized as an uppercase letter and the uppercase counter is incremented.

Algorithm

1. Start the program.
2. Read the input string.
3. Initialize the uppercase counter to zero.
4. Examine each character in the input.
5. Check whether the character lies between A and Z.
6. If yes, increment the uppercase counter.
7. Display the recognized uppercase letters.
8. Display the total number of uppercase letters.
9. Stop.

Implementation



Result

Thus, the program was successfully implemented to recognize and count uppercase letters present in the given input string.

EXPERIMENT 2

Email Address Validation

Aim

To design and implement a program that validates an email address by checking the username, @ symbol, domain name, and domain extension.

Problem Statement

The program accepts an email address from the user and checks whether it follows a valid email pattern.

Algorithm

1. Start the program.
2. Accept an email address.
3. Apply the email validation pattern.
4. Check for a valid username.
5. Check for the @ symbol.
6. Check the domain name and extension.
7. If all conditions are satisfied, display **Valid Email Address**.
8. Otherwise, display **Invalid Email Address**.
9. Stop.

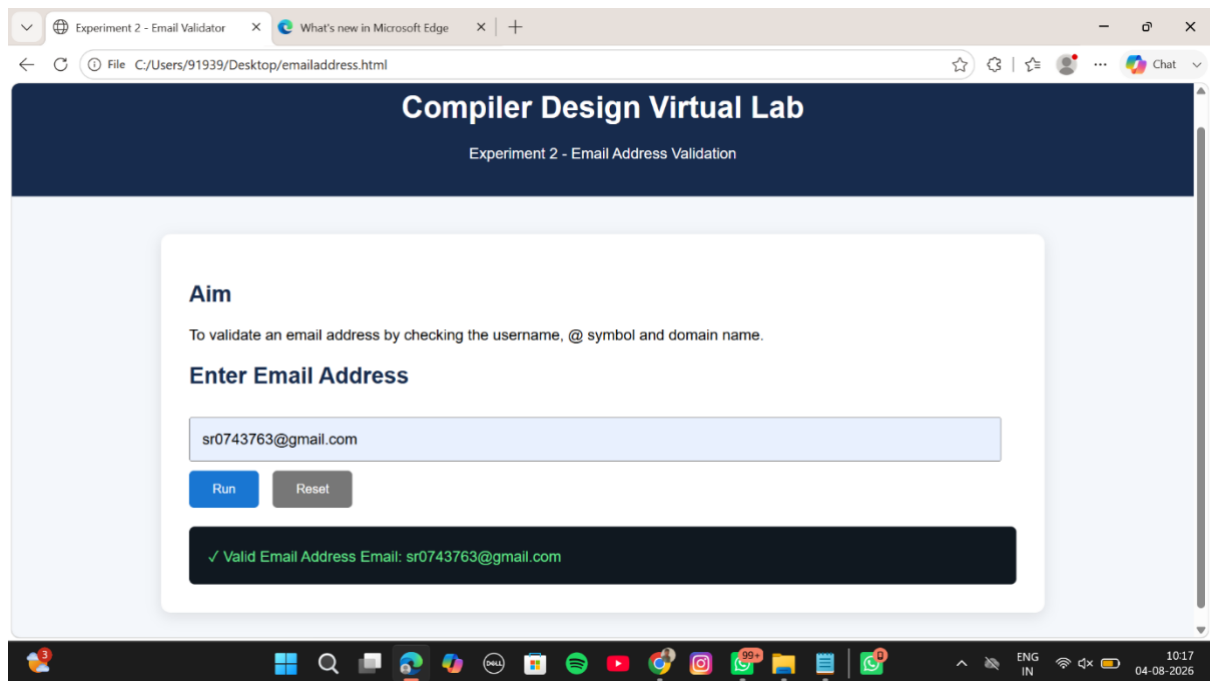
Implementation

Paste your Experiment 2 HTML/JavaScript code here.

Sample Input

student@gmail.com

IMPLEMENTATION:



Result

Thus, the program successfully validates whether the entered email address follows the required format.

EXPERIMENT 3

Date of Birth Validation

Aim

To design and implement a program to validate a date of birth in **DD/MM/YYYY** format.

Algorithm

1. Start the program.
2. Read the date of birth.
3. Check whether it follows DD/MM/YYYY.
4. Validate the day.
5. Validate the month.
6. Validate the year.
7. Check whether the complete date exists.
8. Display **Valid Date of Birth** or **Invalid Date of Birth**.
9. Stop.

Sample Input

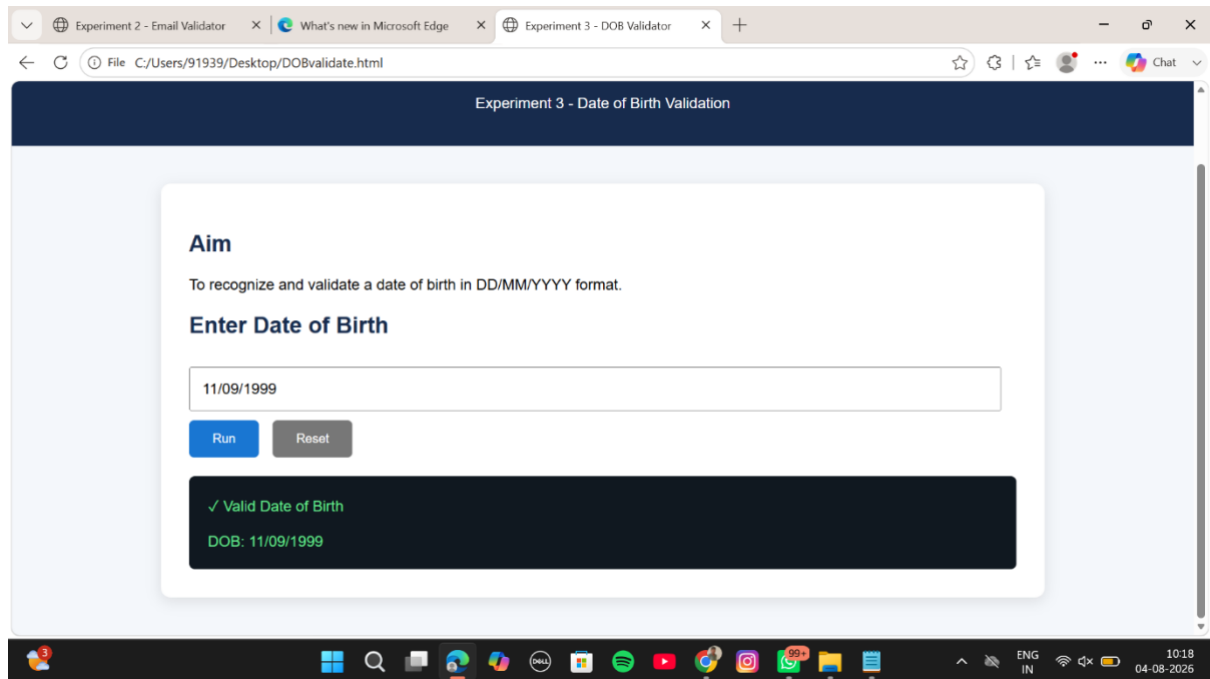
25/12/2005

Sample Output

✓ Valid Date of Birth

DOB: 25/12/2005

Implementation



Result

Thus, the program successfully recognizes and validates the entered date of birth.

EXPERIMENT 4

Token Recognition in a Source Program

Aim

To design and implement a lexical-analysis program that recognizes **keywords, identifiers, operators, separators, and numeric constants**.

Algorithm

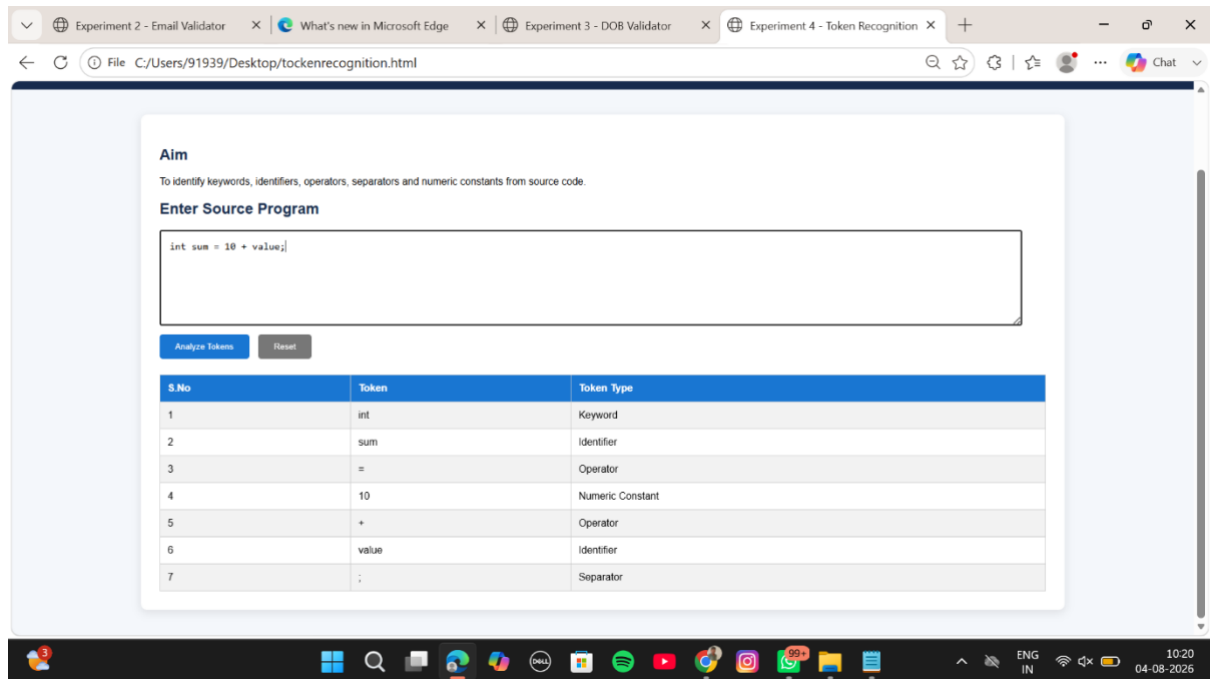
1. Start the program.
2. Accept the source code.
3. Break the source code into individual tokens.
4. Compare each token with the keyword list.
5. Check for numeric constants.
6. Check for operators.
7. Check for separators.
8. Identify remaining valid names as identifiers.

9. Display each token and its corresponding token type.

10. Stop.

Sample Input

```
int sum = 10 + value;
```



Result

Thus, the program successfully recognizes and classifies different tokens present in the given source program.

EXPERIMENT 5

Identification of Vowels and Consonants

Aim

To design and implement a program to recognize vowels and consonants in a given input string and calculate their counts separately.

Algorithm

1. Start the program.
2. Accept a string.
3. Initialize vowel and consonant counters to zero.
4. Examine each alphabetic character.
5. Check whether it is A, E, I, O, U or the corresponding lowercase letter.
6. If yes, increment the vowel counter.
7. Otherwise, increment the consonant counter.

8. Display the identified vowels and consonants.
9. Display their total counts.
10. Stop.

Sample Input

Compiler Design

Sample Output

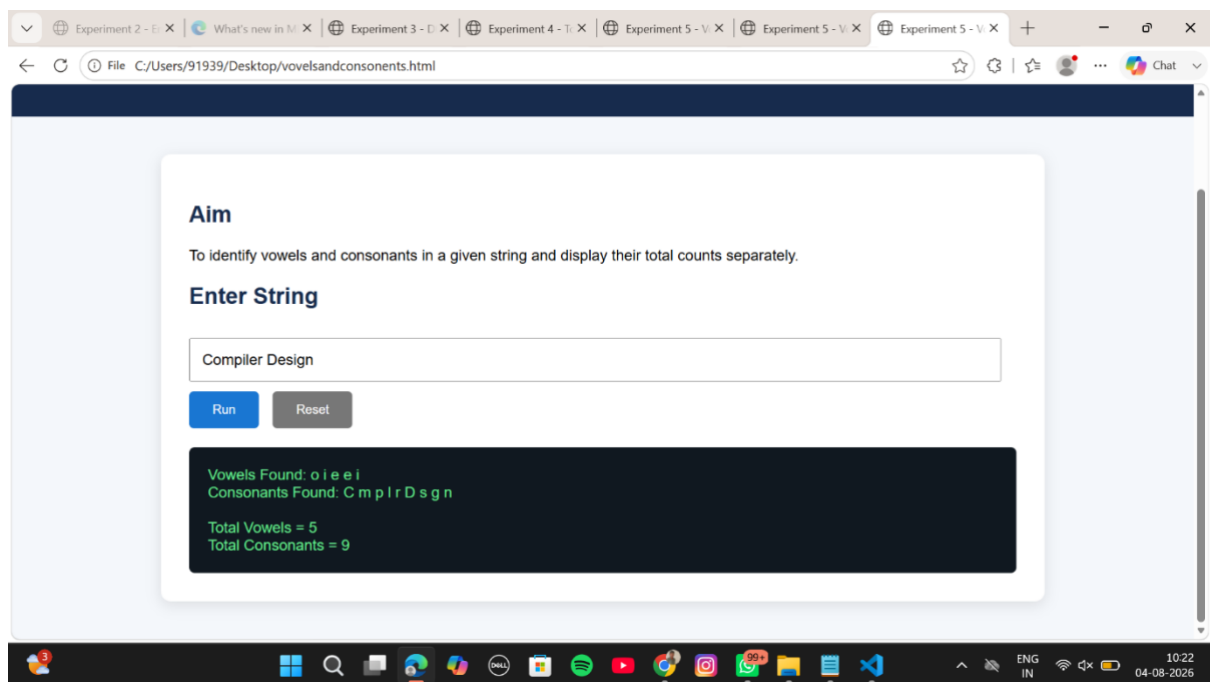
Vowels Found: o i e e i

Consonants Found: C m p l r D s g n

Total Vowels = 5

Total Consonants = 9

Implementation



Result

Thus, the program successfully identifies and separately counts vowels and consonants from the given input string.